

Lov na zaklade

Omejitev časa: 8 s Omejitev pomnilnika: 256 MiB

S pomočjo čarobnega kompasa iščeš davno izgubljene zaklade na veliki mreži $N \times N$ celic. Na $K \leq 3$ različnih celicah mreže so skrite skrinje z zakladi. Tvoj cilj je preprost: najti vse zaklade!

Ko položiš kompas na eno od celic, bo našel najkrajšo tako pot do skrinje z zakladom, ki uporablja le premike levo, desno, gor in dol (z drugimi besedami, do zaklada, ki je najbližji po manhattanski razdalji). Kompas bo nato pokazal smer prvega koraka. Če po najkrajši poti do najbližjega zaklada (ali različnih najbližjih zakladov, če jih je več na enaki oddaljenosti) vodi več različnih prvih korakov, bo kompas vrnil vse te korake. Če pa je na celici skrinja z zakladom, bo kompas to tudi pokazal (namesto smeri prvega koraka).

Ko najdeš skrinjo z zakladom, jo izprazniš, skrinje pa ne moreš odstraniti – pretežka je. Tvoj kompas ne ve, ali je skrinja polna ali prazna; vedno bo kazal proti najbližji skrinji, naj bo prazna ali polna.

Poskusi najti položaje vseh skrinj z zakladi in pri tem uporabiti čim manj poizvedb s kompasom.

Naloga

To je interaktivna naloga. Pri vsakem testnem primeru (torej vsakem izvajanju tvojega programa) bo moral tvoj program rešiti več lovov na zaklade. Z ocenjevalnim sistemom se boš sporazumeval s pomočjo knjižnice, ki so jo pripravili organizatorji tekmovanja. Ta knjižnica vsebuje naslednje deklaracije:

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — to funkcijo pokliči, da začneš naslednji lov na zaklade. V spremenljivki *N* bo vrnila velikost mreže, v *K* pa število zakladov. Če v trenutnem izvajanju ni ostalo nobenega lova na zaklade več, bo funkcija postavila *N* in *K* na -1 ; v tem primeru moraš nato končati svoj program z izhodno kodo 0. Opozorimo na to, da smeš to funkcijo poklicati tudi, če v trenutnem lovu še nisi našel vseh zakladov (npr. če bi rad rešil nalogo za delne točke).
- **enum** {*TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — to so konstante, ki se jih uporablja pri vrednostih, ki jih vrača *Query* (glej spodaj).
- **int** *Query*(**int** *x*, **int** *y*) — če vsebuje celica na koordinatah (*x*, *y*) skrinjo z zakladom, bo ta funkcija vrnila *TREASURE*. Drugače pa vrne vsoto ene ali več izmed konstant *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT*, and *DIR_DOWN*, ki pove, katere smeri pokaže kompas, če ga postavimo na celico (*x*, *y*). (*DIR_RIGHT* = desno, *DIR_UP* = gor, *DIR_LEFT* = levo, *DIR_DOWN* = dol.) Koordinati *x* in *y* morata biti celi števili od 0 do $N - 1$. (Pozor: pri tej nalogi se *y*-koordinate povečujejo od zgoraj navzdol.)

Po tistem, ko *NextHunt* vrne $N = K = -1$, tvoj program ne sme več klicati niti funkcije *NextHunt* niti *Query*. Funkcije *Query* tudi ne sme klicati, preden prvič pokliče

NextHunt. Če se tvoj program ne drži teh omejitev ali pa če pri kakšnem lovu na zaklade izvede več kot 1000 poizvedb ali pa če pri klicu funkcije *Query* poda neveljavne vrednosti parametrov x in/ali y , bo knjižnica prekinila izvajanje tvojega programa in pri trenutnem testnem primeru boš dobil oceno RTE (*run-time error*, napaka med izvajanjem).

Šteje se, da si zaklad našel, če si vsaj enkrat poizvedoval po celici, v kateri ta zaklad leži. Če tvoj program pokliče *NextHunt*, še preden je našel vse zaklade v trenutnem lovu, se to ne šteje za napako, vendar pa se bo poznalo pri točkah (glej spodaj).

Da boš lahko uporabljal knjižnico, vključi v svoj program datoteko `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

To datoteko si lahko preneseš tukaj: `treasurehuntlib.h`.

Na voljo je tudi preprosta implementacija te knjižnice, ki ti bo morda v pomoč pri razvoju tvoje rešitve: `treasurehuntlib-public.cpp`. Da jo prevedeš skupaj s svojim programom, moraš le dodati njeno ime kot parameter pri klicu prevajalnika, npr.:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

če je `foo.cpp` datoteka z izvorno kodo tvoje rešitve.

Implementacija v datoteki `treasurehuntlib-public.cpp` podpira poleg gornjih deklaracij še eno funkcijo: `void InitFromFile(const char *fileName)`, ki prebere seznam lovov na zaklade iz datoteke in ti omogoči, da igraš igro z njimi, namesto da bi generirala svoje naključne love. Več podrobnosti najdeš v datoteki `treasurehuntlib-public.cpp` sami.

Na ocenjevalnem strežniku bomo uporabljali drugačno implementacijo knjižnice, tako da ne delaj nobenih predpostavk glede tega, kako točno deluje implementacija. Smeš pa predpostaviti, da ne smeti po globalnem imenskem prostoru z nobenimi deklaracijami razen zgoraj naštetih (*NextHunt*, *Query* in petero konstant).

Tvoja koda ne sme brati s standardnega vhoda ali pisati na standardni vhod, kajti njiju uporablja naša implementacija knjižnice na ocenjevalnem strežniku za sporazumevanje s preostankom ocenjevalnega okolja.

Vhod

Omejitve

- $2 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- V okviru posameznega izvajanja tvojega programa bo največ 100 000 lovov na zaklade.
- V okviru posameznega lova na zaklade lahko izvedeš največ 1000 poizvedb.

Podnaloge

- 1. podnaloga (10 točk): $K = 1$.
- 2. podnaloga (30 točk): $K = 2$.
- 3. podnaloga (60 točk): $K = 3$.

Točkovanje

Podnaloge je lahko sestavljena iz več testnih primerov (več izvajanj tvojega programa), posamezni testni primer pa lahko obsega več lovov na zaklade. Za potrebe točkovanja se vse love pri posamezni podnalogi ocenjuje skupaj, ne glede na to, kako so bili razporejeni med testne primere. Za i -ti lov na zaklade označimo velikost mreže z $N_i \times N_i$, število zakladov s K_i , število poizvedb, ki jih je izvedel tvoj program, s Q_i , in število zakladov, ki jih je našel tvoj program, s F_i . Poleg tega naj bo S skupno število točk, ki so na voljo pri tej podnalogi. Potem dobi tvoj program pri tej podnalogi naslednje število točk:

- Če je tvoj program vedno našel vse zaklade (torej če je $F_i = K_i$ pri vseh i), je njegova ocena odvisna od $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ točk, kjer je } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Če pa tvoj program ni vedno našel vseh zakladov (torej če obstaja tak i , pri katerem je $F_i < K_i$), dobi

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ točk.}$$

Z drugimi besedami, polovico točk dobiš, če najdeš vse zaklade, drugo polovico pa, če jih najdeš s čim manj poizvedbami. Za vse točke mora tvoja rešitev najti vse zaklade s kvečjemu $11 \lceil \log_2 N_i \rceil$ poizvedbami. Od $11 \lceil \log_2 N_i \rceil$ do $20 \lceil \log_2 N_i \rceil$ poizvedb se število točk zmanjšuje linearno; če pa tvoja rešitev izvede več kot $20 \lceil \log_2 N_i \rceil$ poizvedb, bo dobila le prvo polovico točk (če najde vse zaklade). (Simbola $\lceil \cdot \rceil$ pomenita, da se value $\log_2 N_i$ zaokroži navzgor na najbližje celo število.)

Če gornje formule izračunajo za podnalogo ne-celo število točk, ga bomo zaokrožili na najbližje celo število.

Če se tvoj program sesuje zaradi napake med izvajanjem ali pa se ne drži zgoraj opisanih pravil glede uporabe knjižnice, bo za celotno podnalogo dobil 0 točk. Če torej hočeš dobiti delne točke, ker si našel neko podmnožico vseh zakladov, mora tvoj program lepo končati lov s klicem funkcije *NextHunt*.

Primer

Klic	Vrnjena vrednost
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$